

Collection Framework :-

①

collection :- group of objects

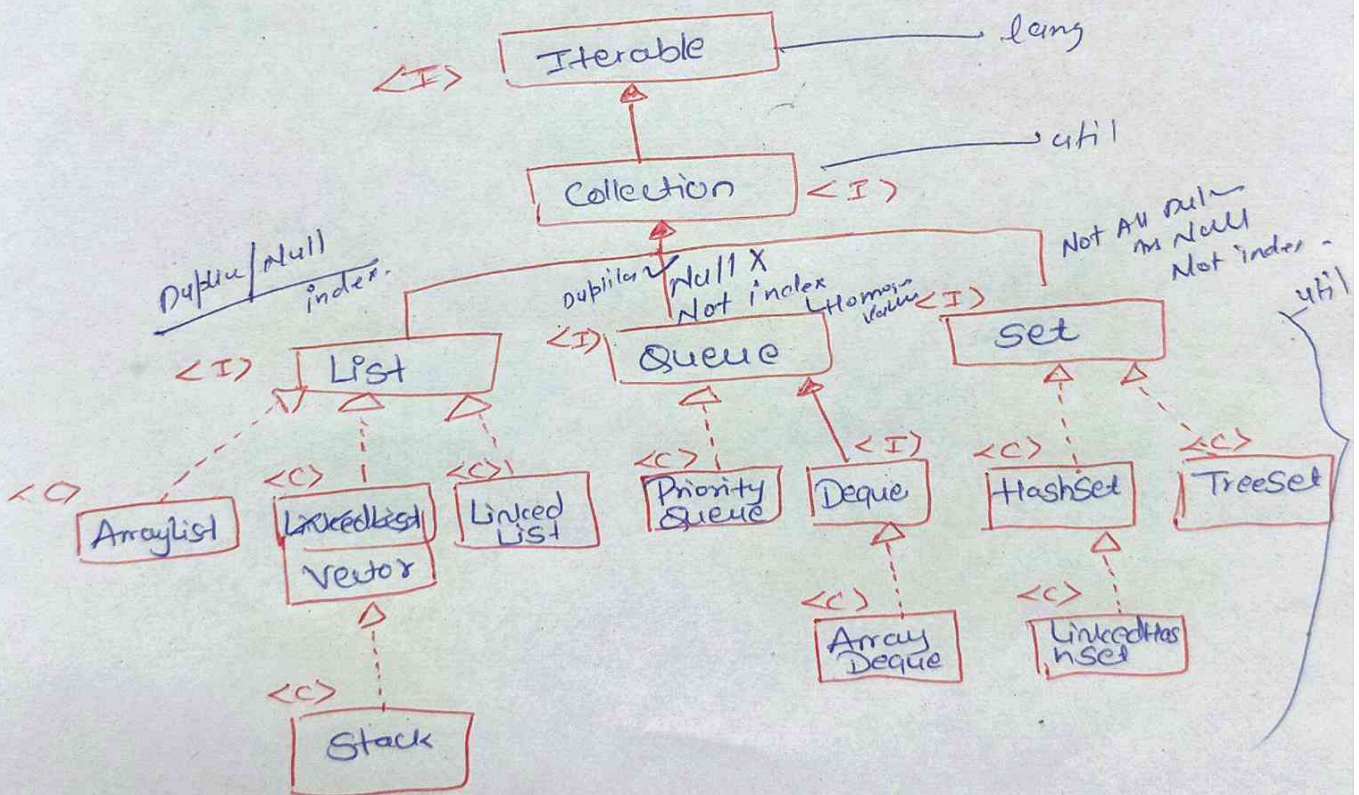
Framework :- readymade architecture

Defⁿ :- It is a collection of predefined classes and Interfaces.

① Collection Hierarchy

② Map Hierarchy

① Collection Hierarchy



① List Inter Iterable (Jdk 1.2) java.lang

It is a interface in which we have one abstract method.

iterator()

Traverse

remove()

hasNext()

next()

• We can traverse only in forward direction with the help of iterator().

Collection :- group of objects, classes & interfaces.

built in methods

CRUD

add elements

add (Object o)

addAll (Object o, collection c)

Remove elements

remove (Object o)

removeAll (collection c)

retainAll (Collection c)

clear()

Search elements

contains (Object o)

~~contains~~

~~index~~ ~~get~~

containsAll (collection c)

Extra methods

size()

iterator()

isEmpty()

toArray()

① List Interface

③

↳ jdk 1.2

↳ java.^{util}~~lang~~ Package

↳ implementation classes

① ArrayList ② Vector ③ LinkedList ④ Stack.

↳ Built-In Methods

add (int index, object o)

addAll (int index, collection c)

remove (int index)

get (int index)

ListIterator() ← Traverse forward as well as backward & also having some methods to manipulate

↳ remove (Object o)

↳ hasPrevious()

↳ previous()

↳ hasNext()

↳ next()

↳ iterator()

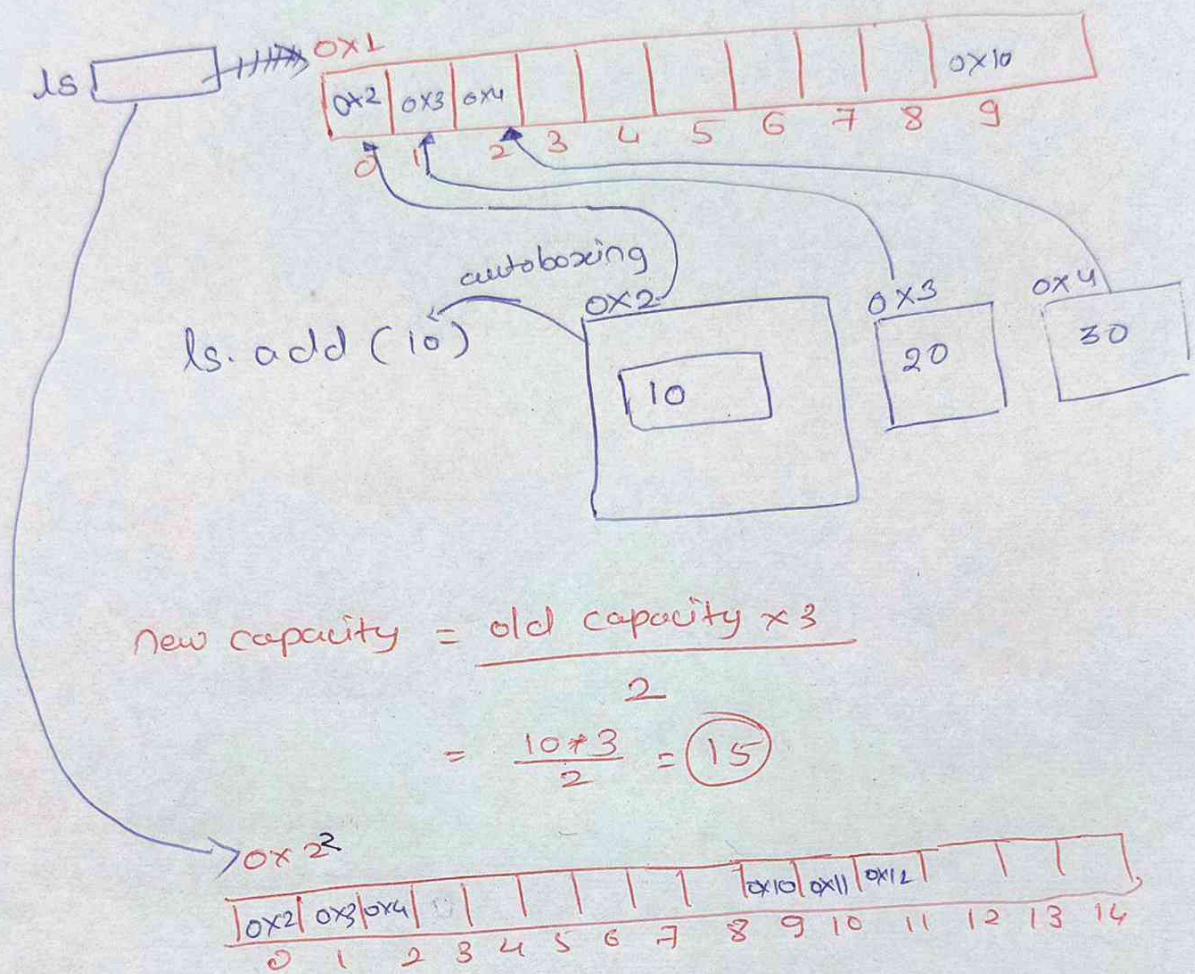
① ArrayList

- java.util, jdk 1.2
- follows insertion order.
- store homogeneous as well as heterogeneous data.
- ds followed is dynamic array.
- size is dynamic
- duplicates are allowed.
- Null values are allowed.
- Not a thread safe
- Initial capacity is 10 & incremental capacity is approximately 50%.

ArrayList < Integer > ls = new ArrayList < > ();

ls.add(10);
ls.add(20);

④



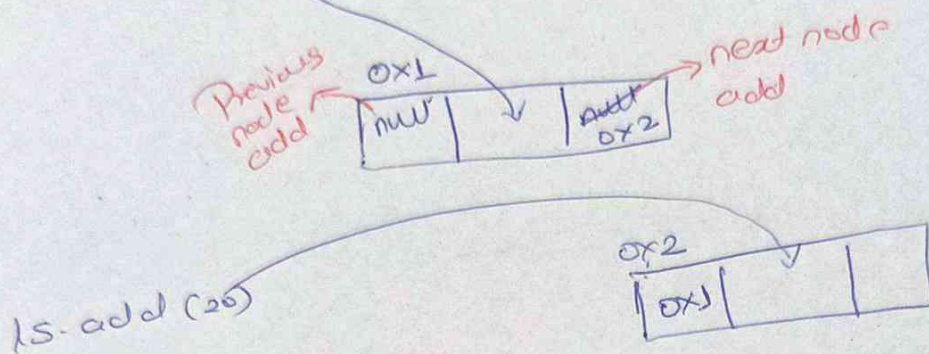
• Retrieving / Traversing

② LinkedList JDK 1.2, Util Package

- Insertion order is maintained.
- Size is dynamic
- allow duplicates
- allow nulls
- no incremental capacity
- DS ← doubly linked list

LinkedList <> ls = new LinkedList <> ();

ls.add(10);



- Insertion or deletion

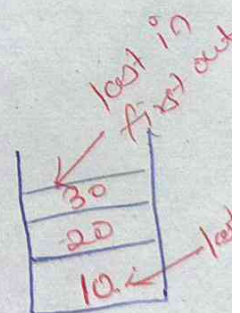
③ Vector

- It is a legacy class
- It is Present from JDK 1.0
- duplicates are allowed
- null value is allowed
- size is dynamic
- It is 10
- Its capacity is 100%
- Thread safe bcz methods are synchronized.
- DS - dynamic array
- Performance is low.
- it follows LIFO

④ Stack

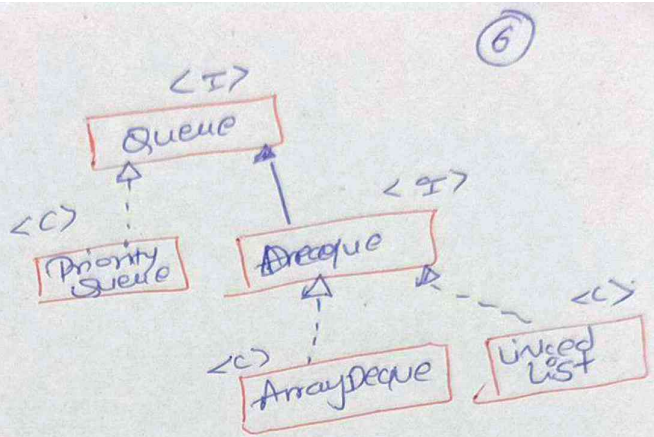
- Stack follows first in last out / Last in

- Push() ← add
- Pop() ← remove element
- Peek() ← Print first element but not remove
- ~~Pop()~~
- Element() ← add create



Queue Interface

- ① Priority Queue.
- ② ArrayDeque.
- ③ LinkedList.



- We can store only homogeneous data.
- Duplicates are allowed
- Null is not allowed
- NullPointerException occurs if you try to insert null values.

Methods of Queue

- ~~add()~~ $\rightarrow$:- To add elements.
- ~~add()~~ $\rightarrow$:- To add elements.
- poll() :- remove head & Print
- remove() :- "
- element() :- return head without removing
- offer() :- Insert element at end
- peek() = return / Print without removing

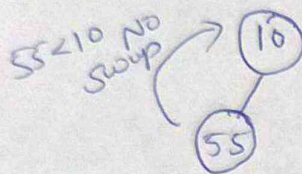
① Priority Queue

- homogeneous ^{as well as heterogeneous} data can be stored.
- duplicates are allowed.
- Null is not allowed.
- Insertion order is not followed.
- Smallest element first.
- DS :- MinHeap $\leftarrow$ complement Binary Tree.

Step 1 :- (10)

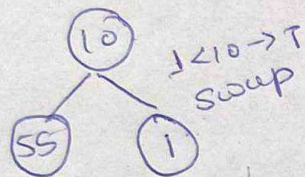
O/P :- [10]

Step 2 :- add(55)



O/P :- [10, 55]

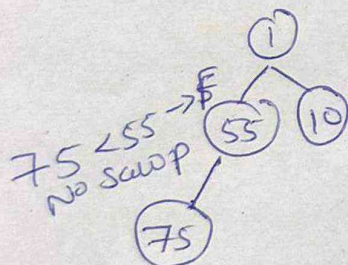
Step 3 :- add(1)



[1 55 10]
0 1 2

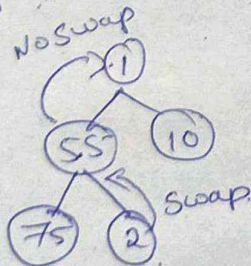
$$2 - 1/2 \\ 1/2 = 0$$

Step 4 :- add(75)



[1, 55, 10, 75]

Step 5 :- add(2)

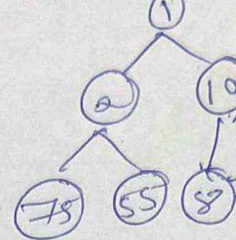


[1 2 10 75 55]

10 | 55 | 1 | 75 | 2 | 8 | 4

7

add(8)

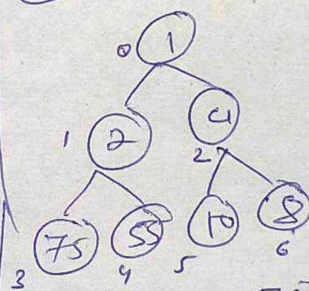
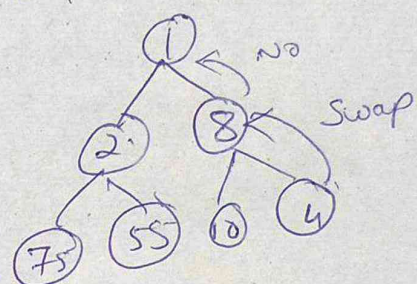


8 < 1 -> No swap

8 < 10 -> T swap

[1, 2, 8, 75, 55, 10]

add(4)



6 - 1/2
5/2
2

[1, 2, 4, 75, 55, 10]
19 8

* Array Deque

(8)

- Follows insertion order
- duplicates are allowed
- Null value is not allowed
- Underline DS is Circular Array

addFirst()

addLast()

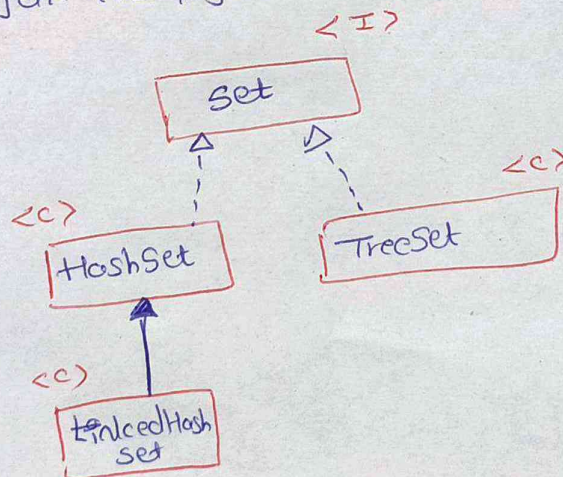
removeFirst()

removeLast()

* Linked List

Same as previous only methods of Queue will be present in this Linked List.

* Set : JDK 1.2, java.util.



- Set interface does not contain any extra methods only we will use methods of Collection.
- When we don't want to allow duplicates then we prefer to go for set.

① HashSet

Note - hashCode() internally perform conversion of data into integer value (hashCode)

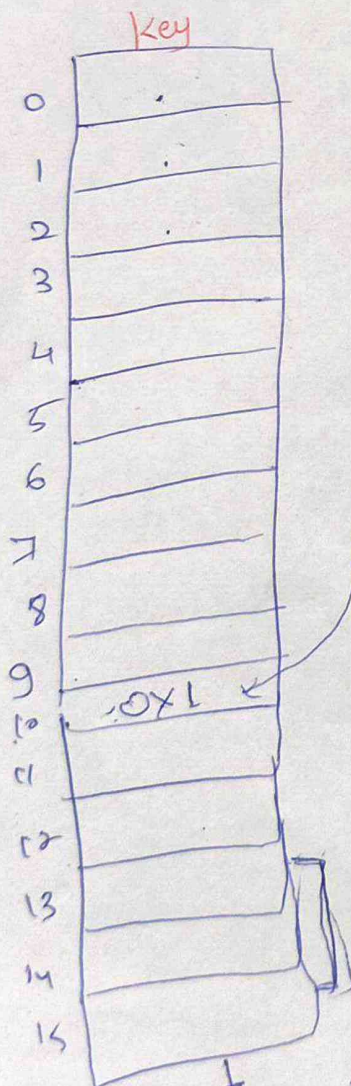
- dynamic in nature
- duplicates are not allowed
- only one null value is allowed. then it will store values in bucket
- Insertion order is not maintained.
- underline DS is HashTable + HashMap
- Initial capacity of HashMap is 16 → load factor 75%
- Initial capacity of HashTable 11.
- Load Factor is 75% (incremental capacity of)
- If you create object of HashSet (HashTable)
- not a index based. → it take size of HashMap to perform hashing.
- we can add both Homogeneous as well as Heterogeneous data.

```
HashSet<> s = new HashSet<>();
s.add(10)
```

10 | 18 | 34 | 5 | 7

hashCode() equals()

Find Particular Bucket index
hash table internally use
this hashCode() & equals()



s.add(10)

0x1
key: 10
Value: null
adjack: null
0x2

obj 0x1
10

0x1 = hashCode
int
int / 0.5 size

initial size
is 11 buckets

10 % 16 = 10
10 / 16 = 0
10 ← rem

0x2
key: 26
Value: null
adjack: null

26 / 16 = 1
26 % 16 = 10
10 ← rem

10.equals(26) → F

new capacity = old capacity * 2 + 1

② LinkedHash Set

(10)

Note :- It we create object of HashTable then only it will perform value % 11, to find the bucket index to store the value.
(Hashcode)
This is the initial capacity of HashTable.

Case 2

If we create object of HashSet then it will perform hashcode % 16 to find bucket index to store the value.
This is the initial capacity of HashMap

② LinkedHash Set

- direct child of HashSet.
- Insertion order is maintained.
- allow only only one null value.
- duplicates are not allowed.
- dynamic in nature.
- Underline DS is doubly linked list and HashMap $\rightarrow$ HashTable.
- we can store Hetrogenous type of data and Homogen as well

③ TreeSet

11

- Insertion order is not maintain
- Size is dynamic
- only we can store homogeneous data.
- null is not allowed.
- duplicates are not allowed.
- o/p will get in fully sorted order.
- underline Datastructure is ~~Red/Black~~
~~Tree~~ (Binary Tree), it will print the
o/p in form of left parent Right
i.e. (inorder traversing)

TreeSet<> t = new TreeSet<>();

[5, 10, 4, 75, 8, 1] → input

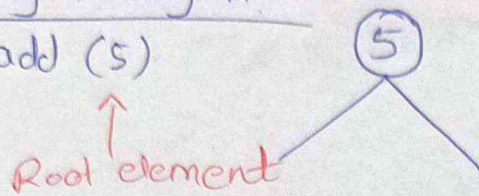
```
t.add(5);  
t.add(10);  
t.add(4);  
t.add(75);  
t.add(8);  
t.add(1);
```

Note:- After it will add all the
Data into Binary Tree which
follow Red-Black Binary tree

→ Internal working is present
in page 122

• Using Binary Tree

Step 1: add (5)



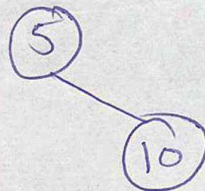
Note:- Internally while adding it will call `compareTo()` 12

Step 2: add (10)

↳ call `compareTo()`

`10.compareTo(5)`

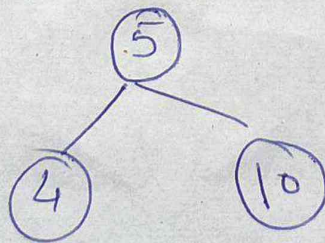
Note:- (current object is greater than passed object
So it will add current object Right to
Passed object)



Step 3: add (4)

`4.compareTo(5)` :- it will compare with root node first

current object is less than passed object so
current object will added to ~~left~~ of root
left or root object.



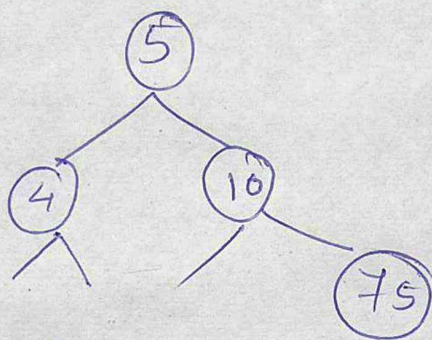
Step 4: add(75)

75.compareTo(5) :- Newly added element will compare first with root.

Current object greater than passed object so it will go right side of passed object. but in the right side already one element is present so again it will compare with the right side present element.

75.compareTo(10) :-

current object is greater so it will get added to right of passed object.



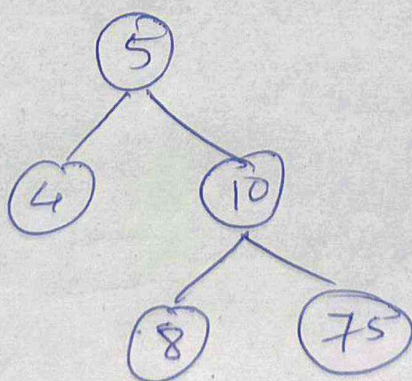
Step 5: add(8)

8.compareTo(5)

↳ go to right

8.compareTo(10)

↳ go to left.



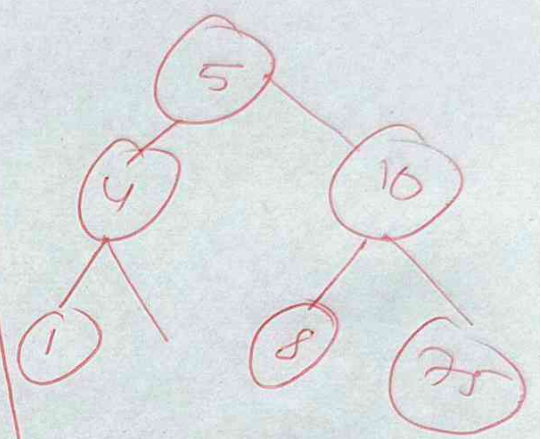
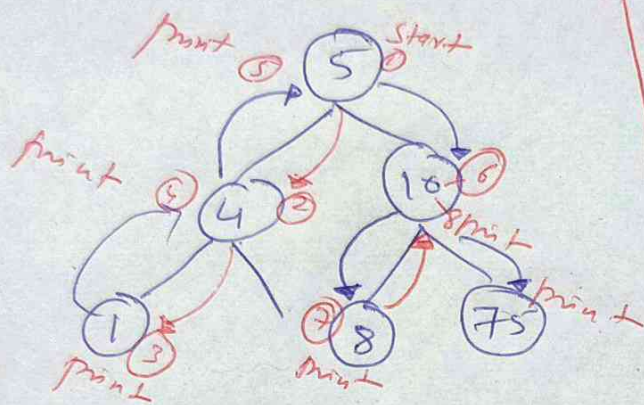
Step 6:- add(1)

1. compareTo(5)

↳ go to left

1. compareTo(4)

↳ go to left



O/p

Now it will print output by following

Inorder Traversing

(left, Parent, Right)

[1, 4, 5, 8, 10, 75]

Using Red-Block Tree

(15)

- Red block Tree is a self balancing Tree
- Two important things we have to learn that is right rotation & left rotation.

Rules of Red-Block Tree

Note :- collision/violation happens for two consecutive red nodes not for black node.

- ① Every node is either red or black.
- ② Root is always black.
- ③ No two consecutive red nodes (no red parent-child) rule collision
- ④ Every path from root to leaf has same no. of black nodes.
- ⑤ Null (leaf) nodes are consider black
- ⑥ Supremost parent (root node) should be black.
- ⑦ newly inserted node is always red.

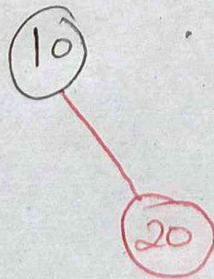
Input :- 10, 20, 15, 30, 25, 5

Step 1 :- Insert 10 (First node becomes root & is black)

(10) ← Root

Step 2 :- Insert 20 (inserted as red default)

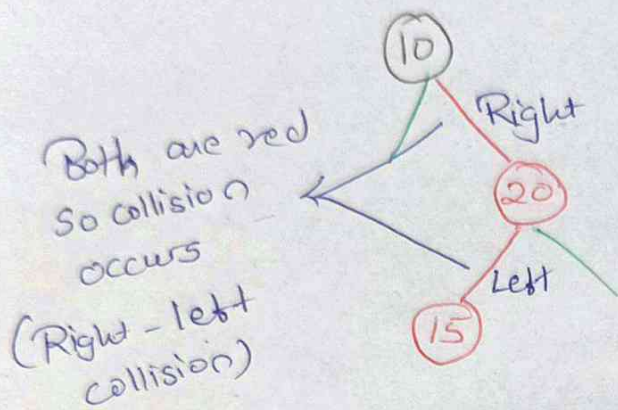
Parent (10) is black → NO collision



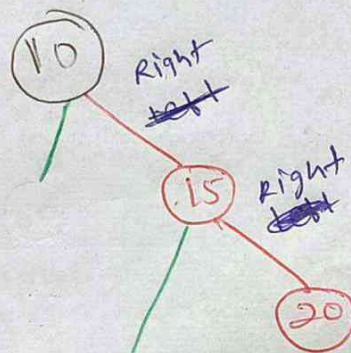
Step 3:- insert 15

- Inserted as right child of 10 $\rightarrow$ left child of 20

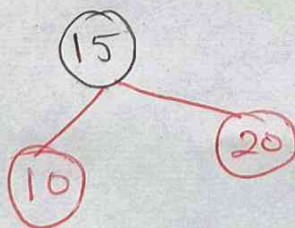
16



This is left-right collision so first it will get swapped.



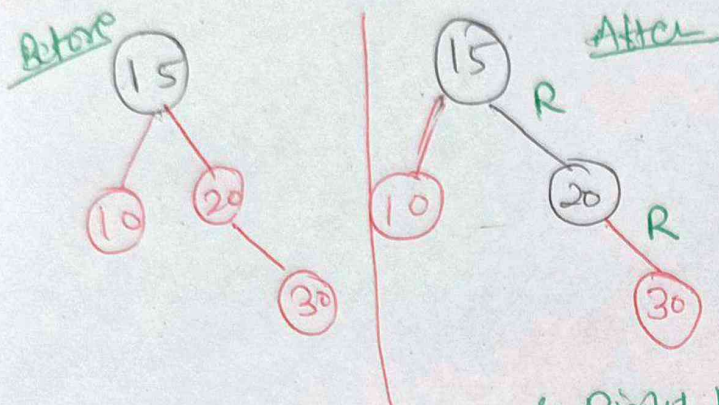
Now it will become ~~Right-Left~~ Right-Right collision so left to Right Rotation will happen (left Rotation)



Step 4: Insert 30

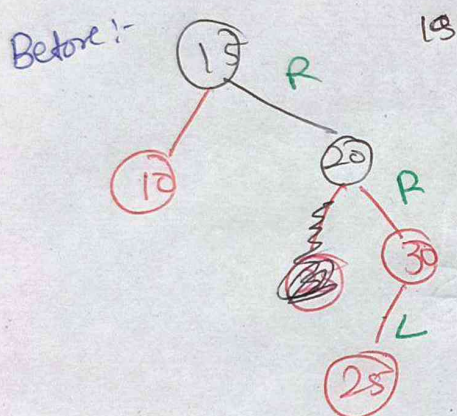
insert 30 to right of 20

Parent = 15 Uncle = 10 both are red



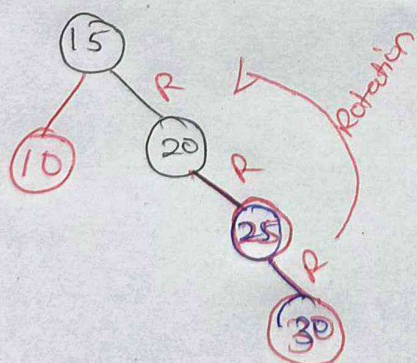
Uncle is not null & Right Right collision is happening change color of Parent node.

Step 5: Insert 25 to left of 30

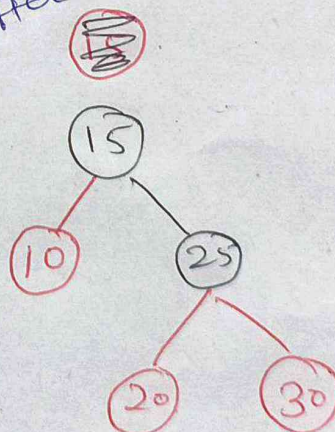


Right Left collision is happening so swap the nodes. & then swap

After:-



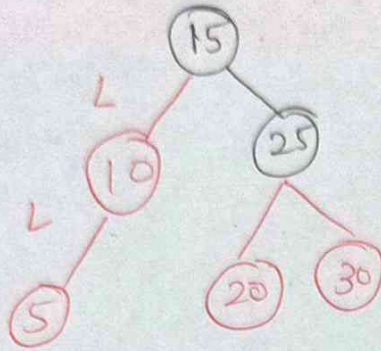
Final (After Left Rotation)



Step 6:- Insert 5 to left of 10

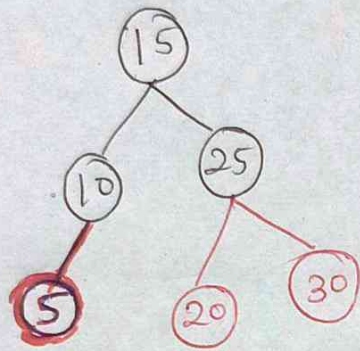
18

Before :-



Left-Left collision is happen & uncle is not null so change Parent node color.

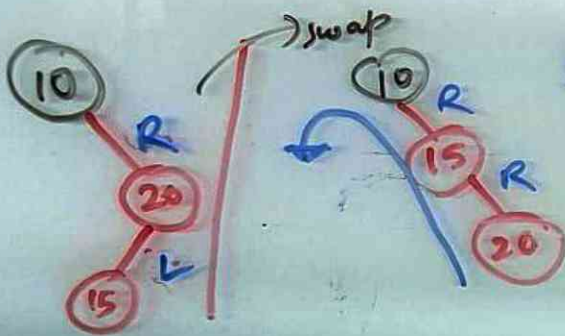
After :-



O/P :- Follow inorder traversing i.e
Left → Parent → Right

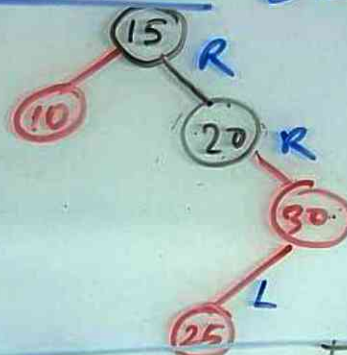
[5 , 10 , 15 , 20 , 25 , 30]

input $\Rightarrow [10, 20, 15, 30, 25, 5]$



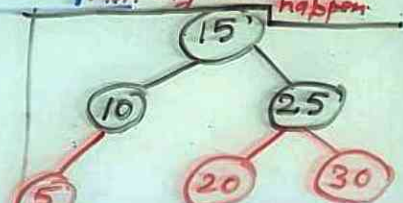
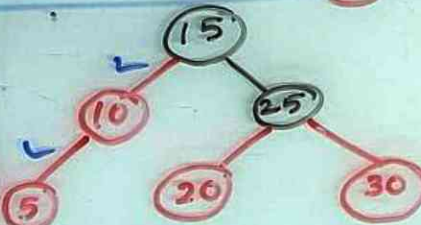
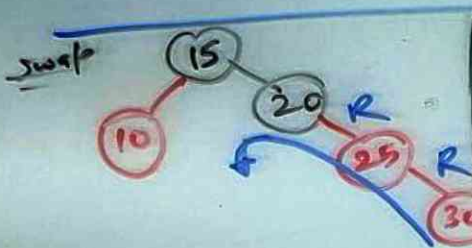
R-R-left rot

L-L-Right Rotation



Note:- if
uncle is present
parent color will
change.

if uncle is not present
then only rotation will
happen.



output:- left, parent, Right
[5, 10, 15, 20, 25, 30]